

Modulo SW Testing

ISW2

ENRICO BARBATANO

Sommario

1. Introduzione e Selezione del SUT	1
2. Progettazione della Test Suite Manuale.....	2
2.1 Category Partition.....	2
2.2 Adeguatezza del Control-Flow.....	4
2.3 Mutation Testing	5
3. Generazione Automatica dei Test	5
3.1 Randoop.....	5
3.2 Evosuite	6
3.3 LLM	6
4. Refactoring Automatizzato	7
4.1 Sintesi delle classi tramite LLM	7
4.2 Creazione di test automatici per le classi rifattorizzate	8
5. Analisi dei Risultati.....	8
6. Integrazione, CI e Minacce alla Validità	10
7. Conclusioni	11
Appendice A.....	11
Appendice B.....	12

1. Introduzione e Selezione del SUT

L'obiettivo di questo report è quello di presentare le tematiche di software testing, software quality assurance e miglioramento della qualità del codice in contesti complessi, dove la manutenibilità e la testabilità sono attributi critici.

La sperimentazione utilizza il progetto Apache Storm, che permette di osservare classi con diversi livelli di complessità e “smell”.

L'intero processo è supportato da un'infrastruttura di continuous integration (CI) basata su gitHub Action e Maven, garantendo la ripetibilità delle build e l'esecuzione sistematica dei test.

L'obiettivo primario della Milestone 4 è caratterizzare il refactoring automatizzato guidato da Large Language Models (LLM), nello specifico Microsoft Copilot, per valutare la sua efficacia nel migliorare la qualità del software senza alterarne le funzionalità originali.

Nello specifico, la ricerca mira a rispondere ai seguenti quesiti sperimentali:

- **Efficacia nella rimozione degli Smell:** Verificare se l'input fornito da strumenti di analisi statica come SonarCloud sia sufficiente affinché un LLM possa eliminare sistematicamente i "code smell" (es. cast ridondanti, classi utility senza costruttore privato, violazioni del diamond operator).
- **Sostituibilità:** Accertare se le varianti generate possano sostituire direttamente le classi originali () mantenendo la correttezza sintattica (compilazione) e la coerenza funzionale (regression testing).
- **Analisi del Delta:** Misurare lo scostamento tra la classe base e le varianti in termini di Code Coverage, Mutation Score e metriche qualitative di chiarezza e manutenibilità.

Selezione del SUT

le classi sono state scelte in base all'algoritmo fornito e senza considerare quelle "banali" (scartando quelle con meno di 70 linee di codice e meno di 5 metodi).

Le classi individuate rappresentano due scenari opposti di complessità:

- **Classe Piccola:** `org.apache.storm.redis.state.RedisKeyValueStateIterator` Selezionata per la sua logica focalizzata (69 LOC), questa classe gestisce l'iterazione su coppie chiave-valore all'interno di uno stato Redis. È stata scelta perché, nonostante le dimensioni ridotte, presenta sfide interessanti legate alla gestione di risorse esterne e alla visibilità dei metodi (molti dei quali protected), rendendola un ottimo candidato per test di unità con mock.
- **Classe Grande:** `org.apache.storm.stats.StatsUtil` Questa classe utility è caratterizzata da una complessità estrema (1709 LOC) e un elevato debito tecnico iniziale (265 smell rilevati da SonarCloud). Gestisce l'aggregazione e la conversione di statistiche vitali per il monitoraggio di Storm, lavorando massivamente su strutture dati annidate. La sua selezione permette di testare i limiti degli strumenti di generazione automatica e degli LLM su larga scala.

2. Progettazione della Test Suite Manuale

2.1 Category Partition

RedisKeyValueStateIterator

Per `RedisKeyValueStateIterator` è stata adottata una metodologia **black-box**, con l'obiettivo di validare il comportamento dell'iteratore indipendentemente dalle strutture dati interne. Questo comportamento è stato dedotto, vista la mancanza di documentazione, dai commenti che spiegano il comportamento dei metodi (**Figura 1**: sintesi comportamento). La classe è stata interpretata come un iteratore responsabile del caricamento incrementale di coppie chiave-valore da Redis tramite `hscan`.

Le **Key Abstractions** individuate sono tre:

- **F1 - Caricamento dati:** capacità di leggere chunk di dati dallo storage Redis.
- **F2 - Riconoscimento terminazione:** capacità di capire se la scansione è conclusa oppure se esistono ancora dati.
- **F3 - Gestione risorse:** corretta acquisizione e restituzione delle connessioni Redis al container.

La **Category Partition** è stata applicata allo **stato osservabile dell'oggetto**, perché la classe possiede un ciclo di vita interno che influenza il risultato dei metodi ([Appendice A](#)). Le categorie principali riguardano:

- stato della scansione Redis;
- stato osservabile dell'iteratore, ad esempio dati caricati ma non consumati;
- esito dell'accesso a Redis, inclusi casi corretti ed eccezioni.

Per controllare in modo deterministico il comportamento dell'iteratore sono stati usati **mock con Mockito**, simulando RedisCommands e ScanResult. Questo ha permesso di testare situazioni specifiche senza usare un'istanza Redis reale.

All'interno delle partizioni è stata applicata la **Boundary Value Analysis**, scegliendo un chunkSize = 2 e testando i valori limite:

- 0: storage vuoto;
- 1: caso minimo;
- 2: chunk pieno.

Come prima misura dell'**adequacy**, la suite è stata valutata con **JaCoCo**, ottenendo:

- Instruction Coverage: **83.00%**
- Branch Coverage: **87.50%**

Questi risultati mostrano che la logica principale di iterazione e caricamento è stata coperta, ma restavano scoperti metodi di supporto come decodeKey, decodeValue e isTombstoneValue.

StatsUtil

Anche per StatsUtil è stata adottata la stessa metodologia **black-box** basata sui commenti (Figura 2:sintesi comportamento da commento). StatsUtil è una utility class molto grande e complessa, usata per trasformare dati grezzi, mappe annidate e oggetti Thrift in statistiche aggregate per bolt, spout, executor e componenti della topologia.

Le **Key Abstractions** individuate sono cinque:

- **F1 - Aggregazione di contatori**: somma di valori numerici su finestre temporali.
- **F2 - Calcolo di medie pesate**: gestione delle latenze in base ai conteggi.
- **F3 - Statistiche Bolt**: calcolo di execute latency e process latency.
- **F4 - Statistiche Spout**: calcolo di complete latency e conteggi acked/failed.
- **F5 - Utility di conversione**: trasformazione di heartbeat e strutture Thrift in mappe Java.

A differenza di RedisKeyValueStateIterator, la **Category Partition** è stata applicata quasi esclusivamente ai **domini di input** dei metodi pubblici statici. Questo perché StatsUtil non mantiene uno stato interno persistente ([Appendice B](#)): il comportamento dipende dai parametri forniti.

Le categorie considerate includono ad esempio:

- **forma dell'input**: A1: input null; A2: input vuoto; A3: input con una sola finestra; A5: input con un solo stream; A6: input con più stream; A7: chiavi presenti in più mappe; A8: chiavi presenti solo in una mappa.
- **valori numerici**: B1: valori positivi; B2: valori uguali a zero; B3: valori mancanti; B5: valori Long; B6: valori Double.
- **medie pesate**: C1: media con conteggio positivo; C2: media con conteggio zero; C5: più stream nella stessa finestra.
- **statistiche bolt**: D1: executed presente; D2: execute latency presente; D3: process latency presente; D4: stream bolt presente.

Per StatsUtil non sono stati usati mock, perché i metodi lavorano principalmente su Map, List e oggetti Thrift istanziabili direttamente nei test. L'uso di Mockito avrebbe aumentato la complessità senza migliorare gli oracoli.

Anche qui è stata usata la **Boundary Value Analysis**, scegliendo input critici come:

- mappe con una sola finestra temporale;
- conteggi pari a zero, per verificare divisioni per zero nelle medie pesate;

Dato l'enorme spazio degli stati della classe, è stata adottata una **scelta unidimensionale**: ogni classe di equivalenza o valore limite viene esercitato almeno una volta, evitando una combinazione esaustiva di tutti i casi. Questa scelta migliora la manutenibilità e la leggibilità della suite.

La prima misura dell'**adequacy** tramite JaCoCo ha restituito valori iniziali molto bassi:

- Instruction Coverage: **9%**
- Branch Coverage: **11%**

Questi risultati sono coerenti con la dimensione e complessità del SUT, che supera le 2000 LOC. La copertura iniziale bassa indica la necessità di una successiva fase di **adequacy growth** per estendere progressivamente la suite.

2.2 Adeguatezza del Control-Flow

Sebbene queste suite siano state progettate seguendo una metodologia **black-box** (focalizzata sul comportamento osservabile e non sulla struttura interna), la loro creazione è stata guidata dai report di adeguatezza di **JaCoCo**. L'intento era quello di coprire zone del codice rimaste "scoperte" o poco esercitate dalla suite iniziale, massimizzando così la **Instruction Coverage** e la **Branch Coverage**.

L'approccio seguito per le classi StatsUtilControlFlowTest e RedisKeyValueStateIteratorControlFlowTest è consistito in una seconda iterazione della suite manuale, aderendo ai seguenti principi metodologici estratti dai documenti di analisi:

- **Identificazione delle aree scoperte:** Utilizzo di JaCoCo sulla baseline per individuare metodi o rami logici non esercitati (es. metodi di supporto o utility minori).
- **Derivazione operativa delle partizioni:** La definizione delle Category Partition in questa fase è stata condotta allo stesso modo della fase precedente, se i metodi non esercitati trattavano partizioni di dati già identificate sono state riutilizzate le medesime classi di equivalenza e gli stessi criteri di selezione.
- **Preservazione dell'approccio Black-Box:** Nonostante la guida fornita dalla coverage, gli oracoli sono rimasti indipendenti dall'implementazione. I test verificano "cosa" fa il metodo (output) e non "come" lo fa, garantendo che rimangano validi oracoli di regressione anche per le varianti rifattorizzate.

Risultati dell'Adequacy Growth

- **RedisKeyValueStateIterator (Classe Piccola):** La suite Control-Flow ha isolato il controllo sui metodi di supporto prima ignorati (decodeKey, decodeValue, isTombstoneValue), introducendo nuove categorie per la decodifica binaria e il riconoscimento dei segnaposto di cancellazione. Questa integrazione ha permesso di raggiungere il **100% di Instruction Coverage** e il **100% di Method Coverage**.
- **StatsUtil (Classe Grande):** Data la complessità massiva, i test Control-Flow si sono concentrati su funzionalità aggiuntive come utility di formattazione stringhe e classificazione dei componenti. Grazie a questo incremento, la **Instruction Coverage è passata dal 9% al 21%** e la **Branch Coverage dall'11% al 26%**.

In sintesi, i test Control-Flow hanno rappresentato il completamento metodologico della suite precedentemente definita, con l'obiettivo di garantire una validazione più profonda dei SUT

2.3 Mutation Testing

Al fine di stimare l'effettiva capacità della test suite nel rilevare errori, è stata condotta un'attività di **Mutation Testing** utilizzando il framework **PIT**. Questa tecnica supera i limiti della semplice code coverage, poiché non si limita a verificare se un'istruzione viene eseguita, ma accerta se gli oracoli (asserzioni) siano in grado di distinguere il comportamento corretto del SUT da versioni artificialmente alterate (mutanti).

RedisKeyValueStateIterator

La prima esecuzione sulla suite completa ha mostrato un'elevata efficacia degli assert, ma l'analisi dei report ha individuato un mutante critico nel metodo **isTombstoneValue**, in cui PIT forzava il ritorno a true.

Il problema risiedeva nel fatto che un test solo positivo non era sufficiente a rilevare l'errore: anche una versione fallata del metodo avrebbe comunque superato il test.

Per risolvere questa debolezza è stata progettata una nuova suite di test, introducendo la classe **RedisKeyValueStateIteratorMutationTest**, focalizzata su casi negativi, cioè sulla verifica che valori ordinari non vengano erroneamente identificati come tombstone.

Questa integrazione ha permesso di migliorare la copertura logica e di eliminare completamente i mutanti generati: tutti e 12 sono stati uccisi, portando il Mutation Score al 100%.

StatsUtil

La prima esecuzione ha generato 415 mutanti con un Mutation Score del 33%, evidenziando come principale criticità l'elevato numero di mutanti non coperti (254), segno che ampie porzioni della classe utility non venivano raggiunte dai test iniziali.

L'analisi dei mutanti sopravvissuti ha messo in luce debolezze negli assert, in particolare sulla gestione di collezioni vuote (sostituzione con strutture immutabili non rilevata) e sulla verifica della precisione nelle medie pesate di latenza, non adeguatamente controllate.

Per colmare questi gap è stata sviluppata la suite **StatsUtilMutationTest**, introducendo verifiche sulla modificabilità delle strutture dati e controlli più rigorosi sui calcoli. A seguito di questi interventi, il Mutation Score è salito al 42%.

Pur restando mutanti non coperti, è stato possibile eliminare 38 mutanti aggiuntivi ed estendere l'esecuzione a nuove aree logiche precedentemente non testate.

3. Generazione Automatica dei Test

Questa sezione descrive l'integrazione di tecniche di generazione automatica volte a complementare la suite manuale, fornendo una validazione aggiuntiva in termini di regressione e copertura strutturale. Sono stati impiegati tre approcci distinti: randomico (**Randoop**), evolutivo (**EvoSuite**) e basato LLM.

3.1 Randoop

L'obiettivo dell'impiego di **Randoop** sulle classi originali è stato quello di ottenere una suite di test di regressione capace di "catturare" il comportamento funzionale della classe **C_0**.

Per entrambe le classi, Randoop è stato configurato per generare test in stile **JUnit 4**, imponendo un limite di 200 test.

RedisKeyValueStateIterator

sulla classe RedisKeyValueStateIterator la generazione diretta ha prodotto una copertura pari a 0%.

Questo è dovuto a due limiti strutturali: da un lato, i metodi principali sono dichiarati **protected**, quindi non direttamente accessibili da Randoop; dall'altro, la classe dipende da componenti Redis/Jedis complessi che non possono essere istanziati automaticamente.

Di conseguenza, Randoop non riusciva ad attivare la logica funzionale della classe e generava solo test che fallivano in fase di costruzione (es. NullPointerException o NoClassDefFoundError), senza eseguire realmente il codice utile ai fini della copertura. Per superare questo problema è stata introdotta una **harness**, che estende la classe, rende accessibili i metodi chiave e fornisce dipendenze controllate:

solo in questo modo Randoop è stato in grado di entrare nei flussi logici e generare test significativi.

Dopo il filtraggio dei test non utili e ridondanti, si è raggiunta una instruction coverage del 57% e branch coverage del 25%.

StatsUtil

Per StatsUtil non è stata necessaria una harness, poiché la classe è composta esclusivamente da metodi statici e non presenta problemi di visibilità o gestione dello stato.

Tuttavia, la complessità della classe e la dipendenza da oggetti Thrift hanno limitato l'efficacia di Randoop: gran parte dei test generati riguardava metodi accessori o casi non significativi, successivamente eliminati tramite filtraggio manuale.

Di conseguenza, la instruction coverage si è fermata al 7% con la branch coverage al 4%, evidenziando come l'assenza di vincoli strutturali non garantisce comunque una buona esplorazione automatica del codice.

3.2 Evosuite

L'adozione di EvoSuite aveva come scopo primario la massimizzazione automatica dell'adeguatezza strutturale (Instruction e Branch Coverage) attraverso una tecnica di generazione basata su algoritmi evolutivi che operano direttamente a livello di bytecode Java.

L'obiettivo era ottenere una suite complementare a quella manuale, capace di esplorare sistematicamente tutti i rami logici delle classi, tuttavia, l'attività ha riscontrato limitazioni significative,

Per una descrizione dettagliata delle criticità tecniche, dei problemi di classpath e delle barriere di compilazione che hanno impedito il pieno successo di questa fase, si rimanda all'analisi approfondita contenuta nel [Paragrafo 6](#).

3.3 LLM

L'uso di Microsoft Copilot è stato finalizzato a valutare l'efficacia degli LLM nella generazione automatica di test complessi tramite diverse strategie di prompt engineering. Sono stati confrontati tre approcci a complessità crescente: **zero-shot**, basato sul solo codice sorgente; **few-shot**, arricchito con esempi di test manuali; e **guided Tree-of-Thought**, che guidava il modello in un processo strutturato di analisi, individuazione dei casi rilevanti e costruzione finale della suite di test.

RedisKeyValueStateIterator

Per la classe `RedisKeyValueStateIterator`, l'approccio basato su LLM si è dimostrato estremamente efficace. La suite generata ha infatti raggiunto il **100% di Instruction Coverage** e il **100% di Branch Coverage**, mostrando una capacità di esplorazione completa del comportamento della classe.

In particolare, il modello è riuscito a cogliere correttamente anche aspetti semantici specifici del dominio, come la gestione dei valori tombstone, producendo test molto vicini per qualità a quelli progettati manualmente.

StatsUtil

Nel caso di `StatsUtil`, i risultati si sono rivelati più contenuti ma sostanzialmente stabili tra le tre strategie di prompting. In tutti i casi, la suite ha raggiunto una **Instruction Coverage del 23%**, mentre la **Branch Coverage** si è attestata al **30%** con **zero-shot** e **guided Tree-of-Thought**, salendo leggermente al **31%** nel caso **few-shot**. Questo mostra che, su una classe molto ampia e complessa, l'aggiunta di esempi concreti può offrire un piccolo miglioramento nell'esplorazione dei rami logici, pur senza determinare un salto significativo nella copertura complessiva. Rimangono inoltre i limiti tipici degli LLM su strutture dati molto annidate e API articolate, che hanno richiesto in alcuni casi piccoli interventi manuali per correggere errori sintattici o riferimenti non validi.

4. Refactoring Automatizzato

4.1 Sintesi delle classi tramite LLM

L'attività di **refactoring automatizzato** ha previsto la generazione di quattro varianti evolutive (c1, c2, c3, c4) per ciascuna classe target. Il processo è stato condotto tramite l'LLM **Microsoft Copilot**, istruito per agire come uno sviluppatore esperto con l'obiettivo di migliorare la manutenibilità del codice originale (c0) sulla base dei rilievi di **SonarCloud**.

Le varianti sono state sintetizzate seguendo una strategia di **prompting incrementale**, fornendo al modello oracoli di test sempre più completi per guidare e vincolare la trasformazione del codice:

- **Variante c1:** Sono stati forniti in input esclusivamente la **classe originale** e il corrispondente **report diagnostico di SonarCloud**. In questa fase l'LLM ha operato senza il supporto di test suite esplicite nel prompt.
- **Variante c2:** Alla configurazione precedente è stata aggiunta la suite di test **Black-Box (Category Partition)**, utilizzata come vincolo funzionale per garantire che le modifiche rispettassero le partizioni di input e gli stati identificati.
- **Variante c3:** Sono stati integrati nel prompt anche i test di **Control-Flow (Adequacy Growth)**, sollecitando il modello a preservare la correttezza del comportamento anche sui rami logici più complessi emersi dall'analisi del flusso di controllo.
- **Variante c4:** È stato fornito il set completo di oracoli, includendo la suite di **Mutation Testing**. L'LLM è stato istruito a garantire che la variante rifattorizzata fosse robusta quanto l'originale anche rispetto ai mutanti generati.

L'efficacia del refactoring ha mostrato risultati divergenti per le due unità, evidenziando come la complessità strutturale influenzi direttamente la capacità dell'LLM di produrre codice pronto per la build.

RedisKeyValueStateIterator

Per questa unità, l'attività ha avuto un **esito positivo**:

tutte le versioni prodotte compilano correttamente all'interno dell'ambiente di progetto e superano con successo i test nelle rispettive suite. Questo risultato conferma la capacità dell'LLM di gestire refactoring lineari (come la rimozione di argomenti di tipo espliciti) mantenendo la stabilità funzionale su classi di piccole dimensioni.

Tabella 1: varianti RedisKeyValueStateIterator

StatsUtil

Nel caso della classe utility massiva, si è riscontrato un **fallimento critico nella fase di build**:

Nessuna delle varianti rifattorizzate risulta compilabile, perciò non è stato possibile eseguire le suite di test, misurare i parametri di adeguatezza dinamica (JaCoCo e PIT) per le versioni di StatsUtil, e definire nuove suite di test generati per ogni versione c_x. Tali parametri sono pertanto classificati nelle tabelle comparative come **N/A** (non applicabili).

Tabella 2: varianti Stats Util

4.2 Creazione di test automatici per le classi rifattorizzate

L'approccio randomico è stato applicato nuovamente su ciascuna versione rifattorizzata utilizzando la medesima configurazione della baseline (con creazione della relativa classe harness).

- **Stabilità dei risultati:** I parametri di adeguatezza per la suite Randoop selezionata sono rimasti invariati rispetto alla classe c_0: la **Instruction Coverage si è attestata al 57.00%** e la **Branch Coverage al 25.00%** su tutte le varianti.

Per la generazione tramite LLM (Microsoft Copilot), è stata impiegata la strategia **Guided Tree-of-Thought (ToT)**, fornendo al modello il codice sorgente già rifattorizzato delle diverse varianti.

- **Performance di copertura:** La suite LLM ha dimostrato un'eccezionale robustezza, mantenendo il **100% di Instruction Coverage** su tutte le versioni rifattorizzate. La **Branch Coverage** si è stabilizzata all'**87.50%** (7 branch su 8), un valore coerente con la logica della classe dove un singolo ramo legato alla condizione di fine dati risulta difficilmente sollecitabile in isolamento.

In sintesi, per la classe piccola, i test automatici hanno confermato il successo del refactoring: non solo le varianti prodotte sono funzionalmente equivalenti alla baseline, ma la loro nuova struttura sintattica continua a permettere una validazione automatica ai massimi livelli di adeguatezza.

5. Analisi dei Risultati

Questa sezione presenta l'**analisi comparativa dei risultati** ottenuti tra la baseline originale () e le varianti rifattorizzate (). La valutazione si articola su due dimensioni: **statica** (rimozione dei code smell e manutenibilità) e **dinamica** (compilabilità, copertura e robustezza ai mutanti).

RedisKeyValueStateIterator

Per la classe piccola, il processo di refactoring ha avuto un **esito positivo su tutti i fronti**, confermando che per unità di codice limitate l'LLM è estremamente affidabile.

- **Compilabilità e Funzionalità:** Tutte le varianti (-) **compilano correttamente** e superano l'intera suite di test. Non è stata riscontrata alcuna regressione funzionale.
- **Analisi Statica (Code Smells):** La baseline presentava un unico smell (S2293 - diamond operator). In tutte le varianti, questo smell è stato **completamente eliminato**, portando il conteggio dei difetti a **zero**. Ciò ha prodotto un lieve ma misurabile miglioramento della chiarezza e della manutenibilità del codice.
- **Metriche Dinamiche (Adeguatezza):**
 - **Coverage:** La suite LLM ha mantenuto il **100% di Instruction Coverage** su tutte le versioni. La suite Randoop si è confermata stabile al **57%**.
 - **Mutation Score:** È l'area in cui si è osservato il successo maggiore. Le varianti e hanno raggiunto un **Mutation Score del 100%**, dimostrando che gli oracoli generati dall'LLM sono in grado di rilevare ogni singola alterazione artificiale del codice.

Tabella 3: valori delta per redikeyValueStateIterator.java

StatsUtil

L'analisi della classe grande rivela un contrasto netto tra l'apparente successo statico e il fallimento dinamico dovuto alla complessità intrinseca del sistema.

- **Compilabilità: Il Fallimento Critico.** A differenza della baseline , **nessuna delle varianti rifattorizzate () è risultata compilabile** nell'ambiente Apache Storm. L'impossibilità di build impedisce qualsiasi validazione funzionale e rende le metriche dinamiche (JaCoCo e PIT) **N/A (non applicabili)** per il confronto evolutivo.
- **Analisi Statica (Code Smells):** Sul piano puramente testuale, l'LLM ha mostrato una capacità impressionante di ridurre il debito tecnico. Il numero di code smell è crollato da **265 ()** a una forbice tra **44 e 49** nelle varianti.
 - La variante risulta la "migliore" staticamente con soli 44 smell residui.
 - Gli interventi hanno rimosso literal duplicati, rimosso cast ridondanti e modernizzato i costruttori Thrift.
- **Chiarezza e Rischio:** Sebbene la chiarezza sia migliorata grazie alla riduzione del rumore sintattico, **il rischio è classificato come alto**. La mancata compilazione suggerisce che l'LLM abbia introdotto "allucinazioni strutturali" o errori nelle firme dei metodi Thrift che superano i benefici della pulizia formale del codice.

Tabella 4: valori delta per StatsUtil.java

In conclusione, l'analisi dimostra che il refactoring basato su LLM è uno strumento potente per la rimozione sintattica dei difetti, ma richiede una **costante supervisione umana** e l'uso di **harness di test robuste** per garantire che i miglioramenti statici non compromettano l'integrità del sistema, specialmente in classi legacy di grandi dimensioni.

6. Integrazione, CI e Minacce alla Validità

Per garantire la stabilità del progetto e la validazione continua dei test sviluppati, è stato configurato un sistema di **Continuous Integration** basato su **GitHub Actions**.

- **Automazione:** Ad ogni operazione di push sul repository GitHub, viene attivato automaticamente un ciclo di build e testing.
- **Esecuzione Test:** vengono invocati i test specifici per i moduli storm-server (classe grande) e storm-redis (classe piccola) tramite il plugin **Maven Surefire**. Per evitare blocchi della build dovuti a instabilità strutturali, la pipeline è stata istruita per escludere esplicitamente i test generati da EvoSuite (!*_ESTest), concentrandosi sulla validazione delle suite manuali, Randoop e LLM.
- **Link repo (ho creato il branch milesto4-refactoring per questo studio):**

<https://github.com/enricobarbatano/storm.git>

Il caso EvoSuite

L'impossibilità di integrare pienamente **EvoSuite** ha rappresentato un limite significativo per l'adeguatezza automatica. Le cause sono state molteplici:

- **Incompatibilità del Plugin Maven:** Il plugin (ver. 1.0.6) richiede la presenza di tools.jar (disponibile solo fino a JDK 8), mentre il progetto Apache Storm 2.7.1 opera su Java 11/17. Inoltre, EvoSuite non è stato in grado di leggere il bytecode Java 11 prodotto dal progetto corrente.
- **Limiti della modalità Stand-alone:** È stato tentato l'uso dello strumento in modalità stand-alone su versioni legacy dell'artifact (storm-redis 2.2.0) compatibili con Java 8. Sebbene la generazione sia stata completata, la suite prodotta si è rivelata **irrilevante**, poiché una volta inseriti nel modulo storm-redis di Apache Storm 2.7.1, i test generati da EvoSuite **non compilano**. Questo è dovuto a incompatibilità tra il runtime di EvoSuite (nato per Java 8) e l'ambiente Java 11/17 del progetto.
-

StatsUti

La minaccia più critica per la classe grande è stata l'**impossibilità di build** delle varianti rifattorizzate dall'LLM.

- **Problemi di Compilazione:** Mentre la baseline compila correttamente, nessuna delle varianti è risultata compilabile nell'ambiente Storm.
- **Cause del Fallimento:** L'LLM ha introdotto **allucinazioni sintattiche** e riferimenti errati alle firme dei metodi, specialmente nel tentativo di gestire le complesse strutture dati generate da **Thrift** e le mappe legacy massivamente annidate presenti in StatsUtil.
- **Impatto sui Risultati:** Poiché la compilazione è un prerequisito per l'esecuzione dinamica, le metriche di **Instruction Coverage (JaCoCo)** e **Mutation Score (PIT)** per le varianti di StatsUtil sono classificate come **N/A (Non Applicabili)**. Ciò ha limitato l'analisi della classe grande a una valutazione puramente statica del numero di code smell (scesi da 265 a circa 44-49) senza poter confermare la neutralità semantica del refactoring.

7. Conclusioni

La sperimentazione ha evidenziato una netta divergenza nell'efficacia degli strumenti di **Software Testing** e **Refactoring automatizzato** in funzione della complessità strutturale del codice.

Per la classe piccola (RedisKeyValueStateIterator), l'integrazione tra test manuali basati su **Category Partition** e generazione tramite **LLM (Guided Tree-of-Thought)** ha permesso di raggiungere l'eccellenza metrica, con il **100% di Instruction Coverage** e un **Mutation Score del 100%**.

In questo scenario, l'intelligenza artificiale si è dimostrata capace di "comprendere" i vincoli semantici (come il riconoscimento dei valori tombstone), producendo suite di test paragonabili per qualità a quelle umane.

Il refactoring è stato altrettanto efficace, eliminando completamente il debito tecnico senza compromettere la stabilità.

Al contrario, la classe grande (StatsUtil) ha rappresentato un limite per l'automazione totale. Sebbene l'LLM sia riuscito a ridurre i **code smell dell'80%** (da 265 a circa 44), è emerso un critico **"compilation gap"**.

Le allucinazioni sintattiche prodotte nel tentativo di gestire API complesse (Thrift) e strutture dati massivamente annidate hanno reso le varianti rifattorizzate non integrabili.

In questo contesto, l'**Adequacy Growth manuale** è rimasto l'unico strumento affidabile per incrementare la confidenza sul sistema (passando dal 9% al 21% di copertura), dimostrando che per sistemi legacy complessi, l'IA deve essere intesa come un **assistente alla produttività** sotto stretta supervisione umana, piuttosto che come un sostituto autonomo del tester esperto.

Appendice A

- **Categoria A - Dati restituiti da Redis:**
 - **a1:** zero elementi (storage vuoto).
 - **a2:** un elemento (minimo non vuoto).
 - **a3:** più elementi.
 - **a4:** lista null (errore di risposta).
 - **a5:** numero elementi uguale al chunkSize (limite superiore chunk).
- **Categoria B - Stato della scansione Redis:**
 - **b1:** scansione conclusa (cursore finale 0).
 - **b2:** scansione non conclusa (cursore non zero).
 - **b3:** multi-chunk (scansione terminata dopo più letture).
 - **b4:** chunk vuoto con cursore non finale.
- **Categoria C - Stato dell'iteratore:**
 - **c1:** nessun elemento disponibile.
 - **c2:** un elemento disponibile.
 - **c3:** più elementi disponibili.

- **c4:** elementi caricati ma non consumati.
- **c5:** elementi caricati e poi consumati.
- **Categoria D - Dati pendenti locali:**
 - **d1:** pendingPrepare e pendingCommit vuoti.
- **Categoria E - Esito dell'accesso a Redis:**
 - **e1:** hscan termina correttamente.
 - **e2:** hscan solleva un'eccezione.
- **Categoria F - Gestione della risorsa Redis:**
 - **f1:** getInstance restituisce oggetto valido.
 - **f2:** returnInstance chiamato dopo lettura corretta.
 - **f3:** returnInstance chiamato dopo eccezione.
- **Categoria G - Decodifica di chiavi e valori:**
 - **g1:** chiave binaria decodificata correttamente.
 - **g2:** valore binario decodificato correttamente.
- **Categoria H - Riconoscimento dei tombstone:**
 - **h1:** valore uguale al tombstone (positivo).
 - **h2:** valore diverso dal tombstone (negativo).

Appendice B

- **Categoria A - Forma dell'input:**
 - **a1:** input null.
 - **a2:** input vuoto.
 - **a3:** una finestra temporale.
 - **a4:** più stream o finestre.
 - **a5:** chiavi sovrapposte nelle mappe.
 - **a6:** chiavi presenti in una sola mappa.
- **Categoria B - Valori numerici:**
 - **b1:** positivi.
 - **b2:** valore zero.
 - **b3:** tipo Long.
 - **b4:** tipo Double.
 - **b5:** valori mancanti (null).
- **Categoria C - Medie pesate:**

- **c1:** conteggio positivo.
- **c2:** conteggio zero (divisione per zero).
- **c3:** media mancante.
- **c4:** conteggio mancante.
- **Categoria D - Statistiche Bolt:**
 - **d1:** executed presente.
 - **d2:** execute latency presente.
 - **d3:** process latency presente.
 - **d4:** stream bolt valorizzato.
- **Categoria E - Statistiche Spout:**
 - **e1:** acked presente.
 - **e2:** complete latency presente.
 - **e3:** stream spout valorizzato.
- **Categoria F - Utility di formattazione:**
 - **f1:** valore null.
 - **f2:** valore decimale positivo.
 - **f3:** stringa più lunga del limite di troncamento (> 200 caratteri).
- **Categoria G - Tipo componente:**
 - **g1:** componentId null.
 - **g2:** componente bolt.
 - **g3:** componente spout (non bolt).
 - **g4:** componente di sistema.
- **Categoria H - Conversioni Thrift:**
 - **h1:** heartbeat null.
 - **h2:** heartbeat valorizzato.
 - **h3:** ExecutorInfo convertito in lista [start, end].
 - **h4:** ExecutorSummary con statistiche null.
 - **h5:** ExecutorSummary con statistiche presenti.
- **Categoria I - Stream di sistema:**
 - **i1:** includeSys = true (mantiene stream "__system").
 - **i2:** includeSys = false (filtra stream di sistema).
 - **i3:** componente specifico richiesto.

- **i4:** componente di sistema filtrato.

The method `loadChunkFromStateStorage()` of type `RedisKeyValueStateIterator<K,V>` must override or implement a supertype method `Java(67109498)`

org.apache.storm.redis.state.RedisKeyValueStateIterator

`protected Iterator<Entry<byte[],byte[]>> loadChunkFromStateStorage()`

Load some part of state KVs from storage and returns iterator of cached data from storage.

Returns:

Iterator of loaded state KVs

`Iterator<Entry<byte[], byte[]>>`

`org.apache.storm.redis.state.RedisKeyValueStateIterator.loadChunkFromStateStorage()`

Figura 1: sintesi comportamento dal commento del metodo

org.apache.storm.stats.StatsUtil

```
public static Map<String,Object> aggTopoExecStats(String window,
                                                    boolean includeSys,
                                                    Map<String,Object> accStats,
                                                    Map<String,Object> beat,
                                                    String compType)
```

A helper function that does the common work to aggregate stats of one executor with the given map for the topology page.

Figura 2:sintesi comportamento da commento del metodo

Tabella 1: varianti `RedisKeyValueStateIterator`

Variante	COMPILA
C0	SI
C1	SI
C2	SI
C3	SI
C4	SI

Tabella 2: varianti `Stats Util`

Variante	COMPILA
C0	SI
C1	NO
C2	NO
C3	NO
C4	NO

Tabella 3: valori delta per redikeyValueStateIterator.java

Delta rispetto a C_0 della stessa suite. Metriche riportate in ogni cella: Δ Code Smells, Δ Instruction Coverage, Δ Branch Coverage, Δ Mutation Score.													
CONF RONT O		C_1			C_2			C_3			C_4		
		Test LLM	Test RND	Test ES	Test LLM	Test RND	Test ES	Test LLM	Test RND	Test ES	Test LLM	Test RND	Test ES
C_0	Test LLM	Smell -1 Instr. cov 0.00% Branch cov. - 12.50% MS +8.33%			Smell -1 Instr 0.00% Branch - 12.50% MS +8.33%			Smell -1 Instr 0.00% Branch - 12.50% MS +8.33%			Smell -1 Instr 0.00% Branch - 12.50% MS +8.33%		
	Test RND		Smell -1 Instr 0.00% Branch 0.00% MS 0.00%			Smell -1 Instr 0.00% Branch 0.00% MS 0.00%			Smell -1 Instr 0.00% Branch 0.00% MS 0.00%			Smell -1 Instr 0.00% Branch 0.00% MS 0.00%	
	Test ES			N/A			N/A			N/A			N/A

Tabella 4: valori delta per StatsUtil.java

Delta rispetto a C_0. Le varianti non compilano, quindi i delta dinamici (coverage/mutation) non sono applicabili													
CONF RONT O		C_1			C_2			C_3			C_4		
		Test LLM	Test RND	Test ES	Test LLM	Test RND	Test ES	Test LLM	Test RND	Test ES	Test LLM	Test RND	Test ES
C_0	Test LLM	Compila : No Smell - 218 Cov/Mut N/A			Compila : No Smell - 218 Cov/Mut N/A			Compila : No Smell - 216 Cov/Mut N/A			Compila : No Smell - 221 Cov/Mut N/A		
	Test RND		Compila : No Smell - 218 Cov/Mut N/A			Compila : No Smell - 218 Cov/Mut N/A			Compila : No Smell - 216 Cov/Mut N/A			Compila : No Smell - 221 Cov/Mut N/A	
	Test ES			Compila : No Smell - 218 Cov/Mut N/A			Compila : No Smell - 218 Cov/Mut N/A			Compila : No Smell - 216 Cov/Mut N/A			Compila : No Smell - 221 Cov/Mut N/A